# Guided financing journey — solution design

Innocent Nortey · October 29, 2026 · Prototype and setup: `README.md`. Labels: **[B]** brief · **[M]** measured in the prototype · **[A]** assumption · **[E]** estimate.

## 1. Problem framing and approach

The brief reads as an application-speed problem; the money says it is a conversion problem. The financing conversation happens while the homeowner decides whether to buy, and when it stalls the contractor loses the job and the lender loses the loan, priced at $200M of origination for 10% more closed deals [B]. The two biggest drop-offs are **product selection** and **income verification** [B]. At both, the homeowner hits something they don't understand (a deferred-interest promo, a failed bank link) and nothing helps them through it. The payment maths isn't the bottleneck.

The solution is a **guided financing journey inside the existing partner portal**, built on one rule: **buttons for decisions** (amount, priority, option), **AI for ambiguity and messy documents** (one hard question about an option, the contractor's estimate, a failed bank link), and **code for every number, disclosure and decision**. The credit engine alone decides credit. It serves the contractor, who needs an answer "in 30 seconds" [B] and never sees the homeowner's income, and the anxious homeowner, who takes over the device for the sensitive steps [A].

**Not built, on purpose:** another payment calculator; a chatbot that runs the application (slow for decisions, and every message becomes a compliance surface); a second income-verification pipeline (Plaid's income products cover pay stubs).

## 2. Proposed architecture

```
Partner portal (React) → portal backend (TypeScript/Node on ECS), new modules:
  finance/      payment maths over the product catalog API (pure code)
  assist/       question box: one product's terms + figures ──→ Claude Sonnet 5.5
  guardrails/   number check · approval language · disclosures · fallback
  application/  estimate extraction + prep agent ──────────────→ Claude Sonnet 5.5
  audit/        one record per borrower-facing message → RDS + raw output in S3
  adapters:     catalog · Plaid · documents (SQS) · credit engine · profiles
```

- **Fits the stack and the team [B].** Modules inside the existing TypeScript backend on ECS, so there is no new service to operate. Each external system sits behind one adapter; the prototype's mocks are those adapters.
- **Model: Claude Sonnet 5.5 (`claude-sonnet-5-5`) via the Anthropic API**, already available [B]. Answers take about 2 s at low effort [M: 1.9–2.3 s] and cost well under a cent [E: ~1.2k input, ~150 output tokens]; preparing an application takes 4.6–9.2 s, mostly extraction [M]. Opus adds latency the use case doesn't need, because code checks every number; Haiku is a candidate for extraction once an eval shows it holds quality. To keep traffic inside AWS, the same model runs on Amazon Bedrock with only a client change.
- **Data boundary:** the server recomputes every figure (it never trusts client numbers); the model never receives homeowner identity or income values, and extraction sees the estimate only.

## 3. AI design decisions

| Where a model is used | Rung | Why not one rung lower |
|---|---|---|
| Question box | Single call | The lowest rung that answers free text |
| Estimate extraction | Single call, JSON-schema output | Regex can't read 15,000 contractors' layouts |
| Application-prep agent | Tool loop: 4 tools; brakes at 6 steps, 45 s, 40k tokens, 3 identical calls | A script would do for 4 tools; the loop pays off as document types are added. Code assembles the fields |
| Bank-failure explanation | Single call | A template would do; it is the first thing to cut |

- **Prompting, not fine-tuning.** Terms change with each catalog version, so a fine-tuned model would bake in stale terms, and there is no labelled data. Prompts are versioned (`ask-v1`) and recorded in every audit record.
- **Context.** Each call is rebuilt from scratch: one product's terms, its computed figures and the question; no other products, history or homeowner data; about 1k tokens. No RAG: the terms fit in a prompt, and retrieval would add a failure mode (the wrong product's terms) without adding knowledge.
- **Hallucination.** Numbers come only from code, and the guardrail rejects any figure code didn't compute. Extraction returns a source quote and confidence per field; code checks the quote is in the estimate and the number is in the quote, validates ranges, computes the financed amount itself, and marks a missing value `needs_input` (the HVAC sample has no total, and none is invented [M]).
- **Out of scope.** Eligibility, approval and credit-score questions are caught in code before any model call and get a fixed redirect; other off-topic questions may only produce `OFF_TOPIC`, which code maps to a template.
- **Evaluation.** Worked examples became tests: payment maths (19), guardrails (48), agent brakes and extraction validation on a fake model (36); `scripts/guardrail-check.ts` blocks 16 of 16 crafted non-compliant outputs [M]; a Playwright run drives both journeys through the real model. Next: a golden set from reviewed audit records (section 6).

## 4. Compliance and safety

- **No implied credit decision [B].** Eligibility questions never reach the model; the prompt forbids eligibility talk; the approval-language check blocks "qualify", "approved", "guaranteed", "no risk", "you can afford", "good shape" and similar. Results say the decision comes from the credit engine, and the demo's engine adapter never passes through an approval.
- **Disclosures on every borrower-facing message [B].** Code attaches APR, term, total cost of credit, total of payments and "Estimated, not an offer of credit or a credit decision" through one shared component. The RESPA line is placeholder copy for Compliance to replace.
- **Guardrail chain.** Number check → approval-language check → disclosures. A failure regenerates once with the reason; a second failure, a refusal or a cut-off answer shows a safe template. Nothing unchecked is shown.
- **Auditable [B]: "exactly what the system said and why".** One append-only record per message: question or event, product and catalog version, allowed numbers, disclosures, raw output per attempt, each check's result, final text, outcome, model, prompt version, latency. The audit screen shows raw and final side by side.

- **Known limits.** The phrase list misses some paraphrase; numbers written as words aren't checked; a dollar figure within $1 of an allowed amount passes. Questions are logged as typed, with SSN-shaped numbers, emails and phones masked; production needs fuller redaction. In the demo, verified income travels with the browser session; production keeps it server-side. v2 adds a narrow implied-approval classifier calibrated on Compliance's labels.

## 5. Rollout plan

- **MVP, about 2 weeks [E]:** Compare options and the question box in the portal behind a feature flag, with the audit log, for 25–50 pilot contractors across two trades. Compliance reviews every answer in week 1, then a sample.
- **Next quarter:** estimate extraction into the application, with a JSM review queue for `needs_review` fields [B: JSM is the operations tool]; bank-failure recovery on real Plaid errors; a homeowner share link; ranking informed by Snowflake funnel data. Later: voice and Spanish.
- **Contractor change management.** No new app. Pilot contractors get a 30-second script ("pick the priority, show three options, let them ask"), a short video and a feedback button on every answer, recruited from contractors with high drop-off so they see the before and after first.
- **Sizing.** Built for the 50-contractor pilot, with a written path to all 15,000 [decided]. The modules are stateless, so ECS adds tasks as sessions grow; model throughput is the limit, so the provider rate tier is raised before each expansion. Peak concurrency, the failure to survive and the spend ceiling are still open questions.

## 6. Monitoring and impact evaluation

**North star: application completion rate** in contractor-assisted sessions (completed ÷ started, weekly, by contractor segment and product). Baseline 62% (100% − 38%) [B], compared with non-pilot contractors over the same weeks so seasonality doesn't read as success.

| Group | Metrics |
|---|---|
| Business | Drop-off at product selection; recovery after a failed bank link; minutes to submit, split by step (baseline 22 [B]); weekly active pilot contractors |
| Guard | Decline and cancellation rates, so completion doesn't rise by pushing homeowners who won't be approved |
| AI quality | Guardrail blocks per 100 answers by reason; safe-template and regeneration rates; share of prefilled fields the homeowner changes; a weekly Compliance sample scored for accuracy and implied approval |
| Operations | Answer latency p95 (target ≤ 5 s [A]); model cost per completed application |

**Feedback loop v1 → v2.** Reviewed audit records, especially regenerations, safe templates and Compliance flags, become golden-set cases. Every prompt, model or catalog change runs that set before release and reports score and cost before and after; safety checks must pass 100%. The model version is pinned, so a provider update is a deliberate change, not silent drift.

**Questions for the lender:** how the 22 minutes splits across steps; which documents staff still review by hand (GoodLeap may already automate some income-document review); and whether homeowners can finish on their own device.